

UNIVERSIDADE DE TAUBATÉ - UNITAU
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS
DISCIPLINA DE REDES II

ROLAND BITTENCOURT
IGOR PORPERTO

RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO DO ALGORITMO CRC32 PARA
VERIFICAÇÃO DE INTEGRIDADE DE DADOS

TAUBATÉ
2026

**ROLAND BITTENCOURT
IGOR PORPERTO**

**RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO DO ALGORITMO CRC32 PARA
VERIFICAÇÃO DE INTEGRIDADE DE DADOS**

Relatório técnico apresentado à disciplina de Redes II, do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade de Taubaté - UNITAU, como atividade acadêmica sobre detecção de erros e verificação de integridade em transmissão de dados.

Professor: Cristiano.

**TAUBATÉ
2026**

RESUMO

Este relatório apresenta a análise e a documentação do projeto *Algoritmo_CRC32*, desenvolvido em Java com Maven para demonstrar a aplicação do CRC32 na verificação de integridade de dados. O sistema simula uma comunicação simples entre emissor e receptor usando um arquivo de texto como meio de transmissão. A classe emissora coleta nome, idade e telefone, monta uma mensagem textual, calcula o código CRC32 e grava a mensagem juntamente com o código verificador. A classe receptora lê o arquivo, recalcula o CRC32 com base na mensagem recebida e compara o resultado com o valor enviado. Quando os valores coincidem, os dados são aceitos; quando diferem, a mensagem é descartada. A análise foi realizada por inspeção do código-fonte, execução do receptor e compilação do projeto com Maven. O projeto demonstra, de forma didática, a importância de mecanismos de detecção de erro em redes de computadores e evidencia que integridade não equivale a criptografia, mas sim à identificação de alterações acidentais durante armazenamento ou transmissão.

Palavras-chave: CRC32. Redes de computadores. Integridade de dados. Java. Detecção de erros.

SUMÁRIO

1 Introdução	4
2 Fundamentação teórica	5
3 Organização do projeto	6
4 Funcionamento da implementação	7
5 Rotas de execução	9
6 Validação e resultados	10
7 Limitações e melhorias	11
8 Considerações finais	12
Referências	12

1 INTRODUÇÃO

Em redes de computadores, a transmissão de dados está sujeita a alterações causadas por ruídos, falhas de armazenamento, problemas de comunicação ou manipulações acidentais. Por esse motivo, é comum que protocolos e aplicações adotem mecanismos de verificação capazes de identificar se uma mensagem recebida é compatível com a mensagem originalmente enviada.

O projeto *Algoritmo_CRC32* foi desenvolvido para representar esse conceito de forma prática. A aplicação cria uma mensagem com dados de usuário, calcula um valor CRC32 e simula o envio por meio do arquivo *dados_enviados.txt*. Em seguida, uma classe receptora lê o arquivo, recalcula o código e decide se a mensagem deve ser aceita ou descartada.

O objetivo deste relatório é documentar o projeto conforme estrutura acadêmica, explicar como o conteúdo de CRC32 foi aplicado e descrever a rota dos dados em cada implementação. A análise considera a organização Maven, as classes Java existentes, o arquivo de transmissão e os resultados obtidos na execução local.

1.1 OBJETIVOS

O objetivo geral é demonstrar a aplicação do CRC32 como mecanismo de detecção de erro. Como objetivos específicos, destacam-se: analisar o papel do emissor, explicar o cálculo do CRC32, apresentar o comportamento do receptor, validar a execução e indicar melhorias possíveis para aproximar o projeto de uma comunicação real em rede.

1.2 METODOLOGIA

A metodologia utilizada foi a análise direta do código-fonte, seguida da compilação com Maven e da execução da classe receptora. Também foram observadas as normas acadêmicas de apresentação aplicáveis a trabalhos escritos, com margens, espaçamento, fonte, sumário, seções numeradas e referências.

2 FUNDAMENTAÇÃO TEÓRICA

CRC é a sigla de *Cyclic Redundancy Check*, ou verificação de redundância cíclica. Trata-se de uma técnica de detecção de erros baseada em operações binárias. A mensagem é interpretada como uma sequência de bits, e sobre essa sequência é aplicado um cálculo que produz um valor de verificação.

O CRC32 usa um resultado de 32 bits. Esse valor acompanha a mensagem transmitida. Quando o receptor recebe os dados, ele executa o mesmo cálculo. Se o CRC calculado for igual ao CRC recebido, considera-se que não houve erro detectável. Se os valores forem diferentes, a mensagem é considerada alterada e deve ser descartada.

É importante observar que o CRC32 não tem finalidade criptográfica. Ele não protege a mensagem contra leitura indevida e não autentica a identidade do emissor. Sua função principal é detectar alterações acidentais, sendo útil em arquivos, protocolos e transmissões em que a integridade precisa ser conferida.

Quadro 1 - Conceitos utilizados no projeto

Conceito	Aplicação prática no código
Mensagem	Texto formado por nome, idade e telefone, separados por vírgula.
CRC32	Código de 32 bits calculado sobre a mensagem completa.
Transmissão	Simulada pela gravação no arquivo <i>dados_enviados.txt</i> .
Recepção	Leitura, novo cálculo do CRC32 e comparação entre os valores.

Fonte: elaborado pelos autores com base na análise do projeto.

3 ORGANIZAÇÃO DO PROJETO

O projeto foi estruturado como aplicação Java Maven. O arquivo *pom.xml* define o grupo *br.com.roland*, o artefato *Algoritmo_CRC32* e a compilação com Java 17. O código-fonte principal está localizado em *src/main/java/br/com/roland*.

A separação das responsabilidades é adequada para fins didáticos. A classe *Main* representa o emissor, a classe *Receiver* representa o receptor e a classe *CacularCRC32* centraliza o algoritmo utilizado pelos dois lados da comunicação.

Quadro 2 - Arquivos analisados

Arquivo	Responsabilidade
<i>Main.java</i>	Coleta dados, monta a mensagem, calcula o CRC32 e grava o arquivo de transmissão.
<i>CacularCRC32.java</i>	Implementa o cálculo CRC32 com valor inicial, tabela de consulta e processamento byte a byte.
<i>Receiver.java</i>	Lê o arquivo, recalcula o CRC32, compara os valores e aceita ou descarta a mensagem.
<i>dados_enviados.txt</i>	Simula o meio de transmissão entre emissor e receptor.

Fonte: elaborado pelos autores com base nos arquivos do projeto.

A rota geral pode ser resumida da seguinte forma:

Usuário -> Main -> CacularCRC32 -> dados_enviados.txt -> Receiver -> CacularCRC32 -> comparação -> aceite ou descarte

4 FUNCIONAMENTO DA IMPLEMENTAÇÃO

4.1 Emissor

A classe *Main* atua como emissora. Ela solicita nome, idade e telefone por meio de *Scanner*. Depois, os campos são concatenados no formato *nome,idade,telefone*. O CRC32 é calculado sobre a mensagem inteira, e não sobre cada campo isolado. Portanto, qualquer alteração em um caractere ou separador modifica o valor final do código verificador.

Após o cálculo, a classe grava o conteúdo em *dados_enviados.txt*. A primeira linha contém a mensagem e a segunda linha contém o valor CRC32. Esse formato cria um pacote didático: dado transmitido mais informação de verificação.

Quadro 3 - Rota interna do emissor

Etapa	Funcionamento
Entrada	O usuário digita nome, idade e telefone no terminal.
Montagem	Os campos são unidos em uma única string separada por vírgulas.
Cálculo	A mensagem completa é enviada para <i>CacularCRC32.calcular</i> .
Envio	Mensagem e CRC32 são gravados em duas linhas no arquivo.

Fonte: elaborado pelos autores com base em *Main.java*.

4.2 Cálculo do CRC32

A classe *CacularCRC32* é utilitária, pois possui construtor privado e método estático de cálculo. Ela inicia o acumulador com *0xFFFFFFFFL*, converte o texto para bytes em UTF-8 e percorre cada byte da mensagem.

Para cada byte, o algoritmo calcula um índice de tabela usando operação XOR e máscara de 8 bits. Em seguida, atualiza o acumulador com deslocamento lógico e consulta à tabela de 256 posições. Ao fim do processamento, aplica a inversão final e limita o resultado a 32 bits por meio de `0xFFFFFFFFL`.

Texto -> bytes UTF-8 -> índice da tabela -> atualização do acumulador -> inversão final -> CRC32

A escolha de UTF-8 é importante porque padroniza a conversão de texto para bytes. Se emissor e receptor usassem codificações diferentes, caracteres acentuados poderiam gerar bytes distintos e produzir CRCs incompatíveis.

4.3 Receptor

A classe *Receiver* lê o arquivo de transmissão com *BufferedReader*. A primeira linha é tratada como mensagem recebida e a segunda é convertida para número com *Long.parseLong*. Depois, o receptor recalcula o CRC32 usando a mensagem lida.

A decisão principal ocorre na comparação entre o CRC recebido e o CRC calculado. Se os valores forem iguais, a mensagem é aceita e os campos são exibidos separadamente com *split(",")*. Se os valores forem diferentes, a mensagem é descartada.

Quadro 4 - Decisão do receptor

Condição	Ação
$cc == cr$	Dados aceitos e exibidos como mensagem íntegra.
$cc \neq cr$	Dados considerados alterados e mensagem descartada.

Fonte: elaborado pelos autores com base em *Receiver.java*.

5 ROTAS DE EXECUÇÃO

A expressão "rota de execução" é usada neste relatório para indicar o caminho percorrido pelos dados dentro do programa. Não se trata de rota de rede real, mas de uma simulação de envio e recepção por arquivo. Mesmo assim, a estrutura permite observar o comportamento esperado em uma transmissão com verificação de integridade.

5.1 Rota de envio

Entrada do usuário -> montagem da mensagem -> cálculo CRC32 -> gravação da mensagem -> gravação do CRC32

Na rota de envio, os dados são produzidos no terminal e enviados ao arquivo. O emissor não valida a integridade depois da gravação; ele apenas prepara o pacote e disponibiliza o conteúdo para o receptor.

5.2 Rota de recepção

Leitura do arquivo -> separação de mensagem e CRC -> novo cálculo CRC32 -> comparação -> aceite ou descarte

Na rota de recepção, a validação acontece antes da utilização dos dados. Esse ponto é essencial: o receptor só exibe nome, idade e telefone quando o código calculado coincide com o código recebido.

5.3 Rota de erro observada

O arquivo atual do projeto contém a mensagem *Igor,35,1299999* e o CRC recebido *3510240782*. Ao executar o receptor, o sistema recalculou *3121230466*. Como os valores são diferentes, a mensagem foi rejeitada.

Igor,35,1299999 + 3510240782 -> Receiver recalcula 3121230466 -> divergência -> mensagem descartada

6 VALIDAÇÃO E RESULTADOS

A validação técnica foi feita por meio de compilação Maven e execução da classe receptora. O comando `mvn -q -DskipTests package` foi concluído com sucesso, gerando classes compiladas e o arquivo *Algoritmo_CRC32-1.0-SNAPSHOT.jar* na pasta *target*.

Listagem 1 - Saída observada no receptor

```
dados recebidos:
mensagem : Igor,35,1299999
crc32 recebido : 3510240782
crc32 calculado : 3121230466
deu erro, os dados foram alterados.
mensagem descartada.
```

Fonte: execução local de `java -cp target\classes br.com.roland.Receiver`.

O resultado confirma que o mecanismo implementado funciona para detectar divergência entre mensagem e código verificador. Quando o conteúdo da mensagem não corresponde ao CRC armazenado, o receptor não utiliza os dados, o que representa o comportamento esperado em uma verificação de integridade.

Quadro 5 - Resultado da validação

Item validado	Resultado
Compilação Maven	Concluída com sucesso.
Leitura do arquivo	Mensagem e CRC foram lidos corretamente.
Comparação de integridade	Divergência detectada e mensagem descartada.

Fonte: elaborado pelos autores com base na execução local do projeto.

7 LIMITAÇÕES E MELHORIAS

O projeto atende ao objetivo didático, porém apresenta limitações naturais de uma simulação simples. O receptor pressupõe que o arquivo sempre possui duas linhas válidas. Se a linha do CRC estiver vazia, ausente ou contiver texto não numérico, poderá ocorrer erro de conversão. Além disso, o uso de vírgula como separador exige que os campos não tenham vírgulas internas.

Outra limitação é que a comunicação ocorre por arquivo, não por rede real. Para a disciplina de Redes II, uma evolução adequada seria substituir o arquivo por sockets TCP ou UDP, mantendo o CRC32 como mecanismo de verificação do pacote transmitido.

Quadro 6 - Melhorias propostas

Melhoria	Justificativa
Validar quantidade de linhas	Evita falhas quando o arquivo de transmissão estiver incompleto.
Tratar <i>NumberFormatException</i>	Permite informar erro claro quando o CRC salvo não for numérico.
Usar formato estruturado	JSON ou CSV tratado evita ambiguidade causada por vírgulas nos dados.
Adicionar testes automatizados	Garante que o cálculo continue correto após alterações futuras.
Simular rede com sockets	Aproxima o projeto do contexto real de transmissão em redes.

Fonte: elaborado pelos autores com base na análise crítica da implementação.

8 CONSIDERAÇÕES FINAIS

O projeto *Algoritmo CRC32* demonstra, de forma clara, como um código de verificação pode ser associado a uma mensagem e usado posteriormente pelo receptor. A divisão entre emissor, algoritmo e receptor facilita a compreensão da rota dos dados e mostra a importância de validar uma mensagem antes de utilizá-la.

A execução observada confirmou o comportamento esperado: quando o CRC calculado no receptor diverge do CRC recebido, a mensagem é tratada como alterada e descartada. Assim, o trabalho relaciona teoria e prática em Redes II, apresentando um exemplo simples e funcional de detecção de erro.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação**. 4. ed. Rio de Janeiro: ABNT, 2024.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023: informação e documentação: referências: elaboração**. Rio de Janeiro: ABNT, 2018.

KUROSE, James F.; ROSS, Keith W. **Redes de computadores e a Internet: uma abordagem top-down**. 8. ed. São Paulo: Pearson, 2021.

ORACLE. **Java Platform, Standard Edition 17 Documentation**. Oracle, 2021. Disponível em: <https://docs.oracle.com/en/java/javase/17/>. Acesso em: 6 maio 2026.

TANENBAUM, Andrew S.; WETHERALL, David J. **Redes de computadores**. 5. ed. São Paulo: Pearson, 2011.

Observação de normalização: este relatório foi formatado com página A4, margens de 3 cm à esquerda e superior e 2 cm à direita e inferior, fonte Times New Roman 12, espaçamento 1,5 no texto, referências em espaço simples, seções numeradas e elementos pré-textuais, textuais e pós-textuais.